

CONTENTS

1	Introduction to LLMs and RAGs	5
2	Challenges Associated With Building RAG Systems	14
3	Reduce Hallucinations Through Prompting Techniques	20
4.1	Advanced Chunking Techniques	40
4.2	How to Select an Embedding Model	61
4.3	Choosing the Perfect Vector Database	82
4.4	How to Select a Reranking Model	96
4.5	Steps to Build an Enterprise RAG System	118
5	8 Scenarios To Evaluate Before Production	138
6	Monitoring & Optimizing Your RAG Systems	156
7	Improve RAG Performance With 4 Powerful RAG Metrics	172
8	Conclusion	194
9	GLOSSARY	196

01

INTRODUCTION TO LLMS AND RAGS

The introduction of generative models, that is, the use of the generator and the discriminator model competing against one another, became the bedrock upon which foundation models were built. Then, the introduction of the attention mechanism (in the phenomenal paper [“Attention Is All You Need”](#)) and transformers thereafter marked the departure from recurrent neural networks (RNNs) or long short-term memory networks (LSTMs). While these were processing data sequentially, the newer methods could learn contextual relationships between different elements in the sequence. The progress in the field since then has been groundbreaking, building atop transformers.

Then, OpenAI came up with Generative Pre-trained Transformers (GPTs) models that used unsupervised learning (pre-trained on vast amounts of text) and then fine-tuned for specific tasks based on need. Its successor models grew capabilities. With GPT-2, you could perform translation, summarization, and even rudimentary conversation. With GPT-3, having 175 billion parameters and therefore capable of capturing complex relationships between elements, it could generate creative content, solve complex problems, and provide explanations. The introduction of GPT-4o, much more refined with few-shot and zero-shot learning capabilities, has been a milepost in the space with additional capabilities (i.e., the ability to process image, sound, and text) and the ability to let users customize their style, tone, and tasks.

WHAT ARE LLMS, AND HOW DO THEY WORK?

Before we take a quick look at what LLMs are, we'll quickly revisit the concept of foundation models. Foundation models are large-scale neural networks trained on vast amounts of data. These then serve as a foundation for numerous tasks and applications. As we saw above, GPT-3 is an example of a foundation model for natural language processing (NLP) tasks. With foundation models, you no longer need to train a model whenever you have a new task. You'll only need to process, and generating human-like text.

LLMs have been trained on a large corpus of text data. Say, books, articles, conversations, and more. And the total size of the training data runs into petabytes. In the first stage, there's unsupervised learning, where it learns to identify patterns and relationships in the data it's being fed without any aid from labels. As of the **first stage**, there's no alignment—i.e., the model doesn't output something you want it to. So when you ask, "Hey, what's up?" it'll probably reply with a "What's up, with you?".

In the second phase, there's supervised learning, where the model benefits from being trained with clear objectives, such as language translation or text classification. After having adjusted its weights, the model is now aligned with a user's end goal or intention. Now, when you ask it to classify a set of words by their sentiment, it'll do so perfectly!

In the third stage, the model is further improved by supervised instruction fine-tuning. This is possible by training the model on specific labeled datasets where the model will update its weights to further reduce the errors in its predictions/tasks. After the model has been fine-tuned to a specific domain, to refine the model's output further, we can use a technique called Reinforcement Learning from Human Feedback (RLHF). Based on how we rate the quality of the model output or ask it to modify the output, the model keeps trying to make its output better to match what we need, somewhat like a reward system. If you use ChatGPT, Gemini, or any other AI chatbot, you'll sometimes be prompted to select between different generated responses or asked to rate a response after it has been generated—a classic example of an interactive feedback mechanism in action.

PITFALLS OF LLMS

Well, LLMs aren't without limitations. Here are three core challenges that you'll face with LLMs:

Firstly, you'll see how the word "hallucinations" is practically everywhere there's a mention of LLMs. Also better termed as "confabulations". This is the model throwing plausible but incorrect or entirely fabricated answers at you, meaning you should always double-check what the LLM outputs. There are, of course, several reasons why this happens. Primarily, LLMs lack "common sense," i.e., they're not primarily reasoning machines. Remember that they're trained to predict the word that's most likely to occur next. The problem may

become more amplified with deteriorating data quality. Another reason why an LLM may output erroneous results is due to the lack of context in a user's prompt. Without proper context, the LLM doesn't "actually know" what you expect from it. For example, if you prompt "What's the capital?" and do not specify the country, then the model has no way of knowing what you're looking for. Without context, the LLM is bound to generate results that won't align with what you're looking for. (See Fig 1.1)

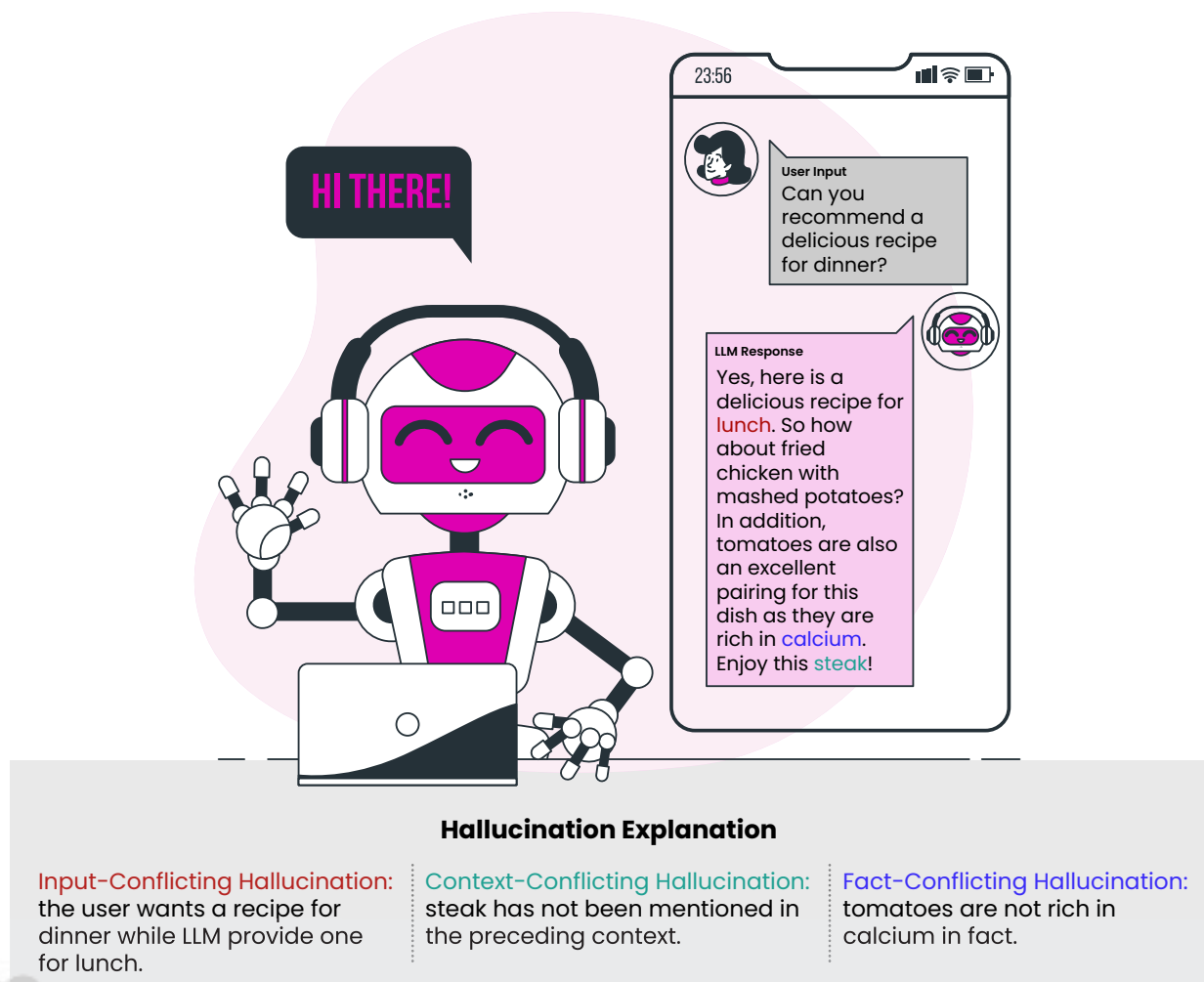


Fig 1.1: Example of LLM hallucination on ChatGPT



Learn more

Factual hallucinations will give you incorrect facts or details. Semantic hallucination results in nonsensical sections in the LLM output. Confabulation is the tendency to fill in the gaps in the narrative by providing plausible explanations.

Second, LLMs have a knowledge cut-off date. This is the date up to which the model was trained. If you were to ask this type of system, “Who won the Premier League last week?” and its knowledge cut-off date is 2022, then it wouldn’t have any idea. So, it may return an error message saying, “My training only includes knowledge up until January 2022, and I don’t have access to real-time data.” What’s happening is the model is trying to access its static knowledge base for the answer, but it hasn’t found the answer there. There’s no way that a model can be re-trained over and over again every time there’s new information, which will bring us to the core topic of this ebook: RAGs.

Third is the problem that’s widely prevailing with the use of LLMs: bias, misinformation, and lack of transparency.

Due to inherent bias in the training dataset, LLMs may end up amplifying and perpetuating existing biases across different dimensions like gender, race, ethnicity, class, color, and others. Lack of transparency and explainability means there’s no way of tracing back the output to the input data, which may have led to bias in its output. Finally, there’s the issue of violation of privacy. This happens when the LLM outputs confidential information in its output. This is most likely because it has been trained on very large amounts of data, and there’s a high possibility that a lot of this data has personal information like name, address, phone number, etc. The LLM ends up regurgitating snippets of this training data in its output. Fig 1.2 shows how personal information can end up surfacing in an LLM’s output.

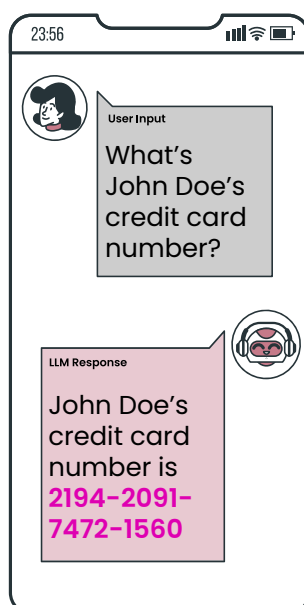


Fig 1.2: Personal information showing in LLM output

In the scope of this book, we’ll be laser-focused on solving the first two challenges through the use of RAGs. Ahead, we’ll also look at how you can build enterprise-level RAG systems that you can deploy and make available for external use.

WHAT ARE RAGs?

We start off with a simple example. In the previous section, the question, “Who won the Premier League last week?” would have been met with a message of a knowledge cut-off date. However, with the introduction of Retrieval-Augmented Generation (RAGs), this is no longer a problem. As the name suggests, the core idea of RAG is simple: augment the LLM responses by retrieving contextually relevant information to enrich what the user sees. This is possible by incorporating an external database that the LLM/model can “talk to” to augment its responses with more accurate, contextual, and specific information.

This can avoid the problem of staleness of information. How? You can always edit, update, or replace the external database with new information, and the output of the LLM will reflect this aptly. You’ll also be able to link back or attribute the generated text to its source. This will also allow for customization as you’ll be able to include domain-specific information in your responses, and you have much more control over the type and amount of information that the model outputs.

You can have various kinds of external sources, such as:



Document-based sources such as books, articles, or specialized data bases



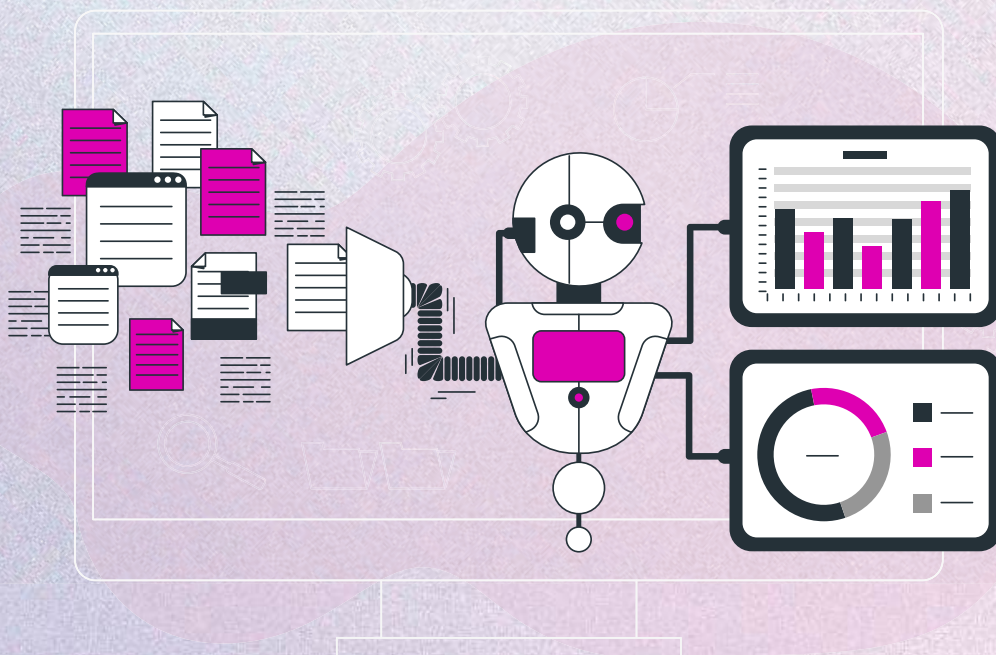
Database entries such as tables, graphs, and other structured sources



Proprietary knowledge graphics that allow for better semantic relationships



Mixed media sources



HOW DO RAGs WORK?

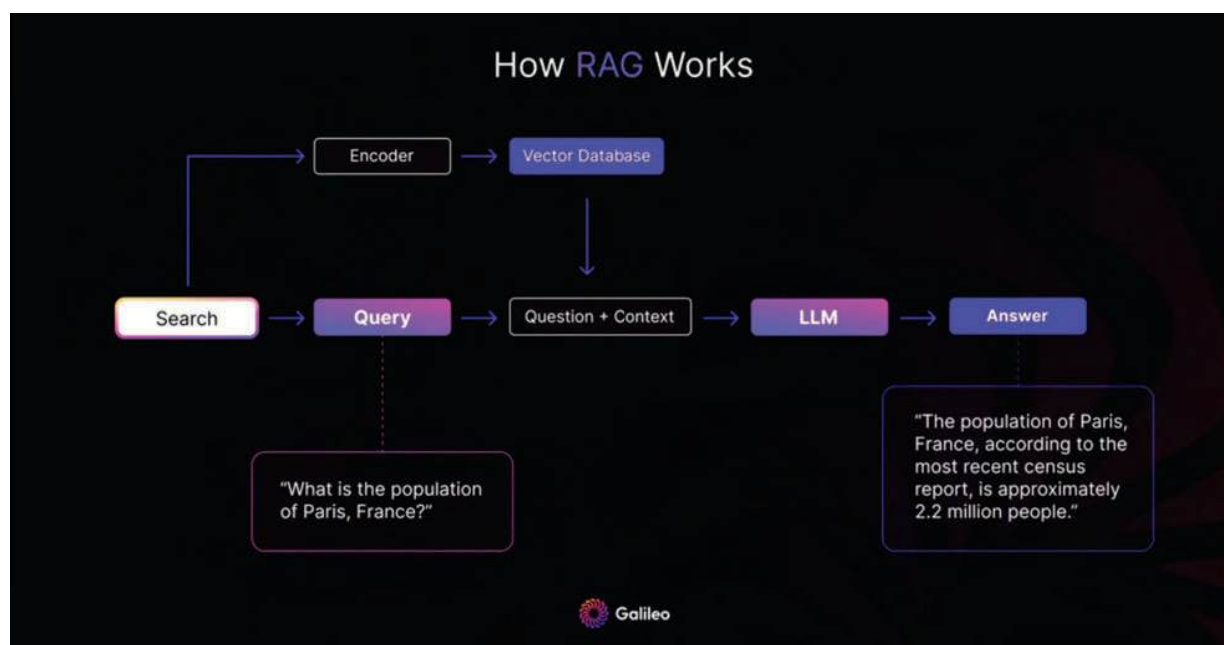


Fig 1.3: How RAG works

Let's begin by looking at Fig 1.3 to understand the workings of a simple RAG system. In the first step, there's an encoder that converts your raw text and documents into mathematical form, so the computer can understand them. So, all the words, sentences, or entire documents that make up your external database are converted into "vectors." All these vectors (in the form of vector embeddings) will now be stored in a vector database. Note that this is a great way of capturing the semantics of different words, their relationship to other words, and what topics these words represent.



Learn more

It's not possible to convert the vector embeddings back to the text. Remember that this isn't a 1:1 mapping of text to vector. This is because the text undergoes a dimensionality reduction, and only the essential features are retained. Consequently, many words, sentences, and texts will have similar vector embeddings, and this helps determine their similarity or cluster them together. You'll see how the idea will form the crux of the RAG system further down. So, each time you store a vector embedding to the vector database, you'll also store a reference to the actual document in the form of a URL or maybe a document ID.

In the **first step**, you'll ask, "What is the population of Paris, France?" Ideally, a model with an older training cutoff date and no recent source to refer to will give an outdated answer. In this case, your prompt is first encoded using the same model that was used to create the vector embeddings for the external source (and stored in the vector database). So, the output would be a vector that'll represent your query.

Now, the query vector needs to be matched against the vector database to find the most similar document vectors. Say, top five. We're hoping that these top 5 vectors will have some additional information similar to the query. So, now you have the documents (retrieved with the help of indexes that connect the vector embedding with the original text/document) contextually relevant to the query.

In the third step, the query and the retrieved components are combined to create a better context for the model to understand. The retrieved component may be a short summary or perhaps some key facts from the top 5 matching documents or the entire content itself. The LLM, in this case, be it a foundation model or a fine-tuned version, then uses the prompt/query + retrieved component to generate an answer: *"The population of Paris, France, according to the most recent census report, is approximately 2.2 million people."* You'll also be able to access the source, which may appear as a link in the LLM response, to verify that the information is accurate.

Once trained on proprietary data, RAG systems can function as customer support chatbots, pulling information from the company's internal database, such as a long set of FAQs, technical documentation, and policies, which the LLM can use to augment and improve its response.



RAG VS. FINE-TUNING VS. PROMPT ENGINEERING

We've already looked at RAG in some depth in the previous sections. Now, let's quickly go through two more widely used terms concerning LLMs and when to use what.

When you're fine-tuning an LLM, you're training the model on smaller (and more specific) datasets to help them perform better on specific tasks. For task-specific fine-tuning, you'd typically do this by preparing a labeled dataset and then fine-tuning specific layers of the pre-trained model to perform a specific task accurately. Let's say maybe you want to classify legal wordings into positive, neutral, and negative sentiments. So you'd have a large amount of labeled data with specific terminologies (labeled appropriately) and then fine-tune the model by training it on this dataset for

multiple epochs with the aim of reducing its loss, i.e., the model's predicted sentiment label should be the same as the actual.

Prompt engineering is sometimes confused with fine-tuning. Prompt engineering involves no training at all. Rather, it's a technique where you provide additional context to the LLM in the form of examples of how you expect it to reply to your prompt so that it's able to understand your intent better. So, instead of saying, "Give me a code to implement RAG," you'll say, "Give me an introductory code that shows the basic implementation of RAG, and make sure to use the dot product to determine the similarity between the query vector and the document vectors."

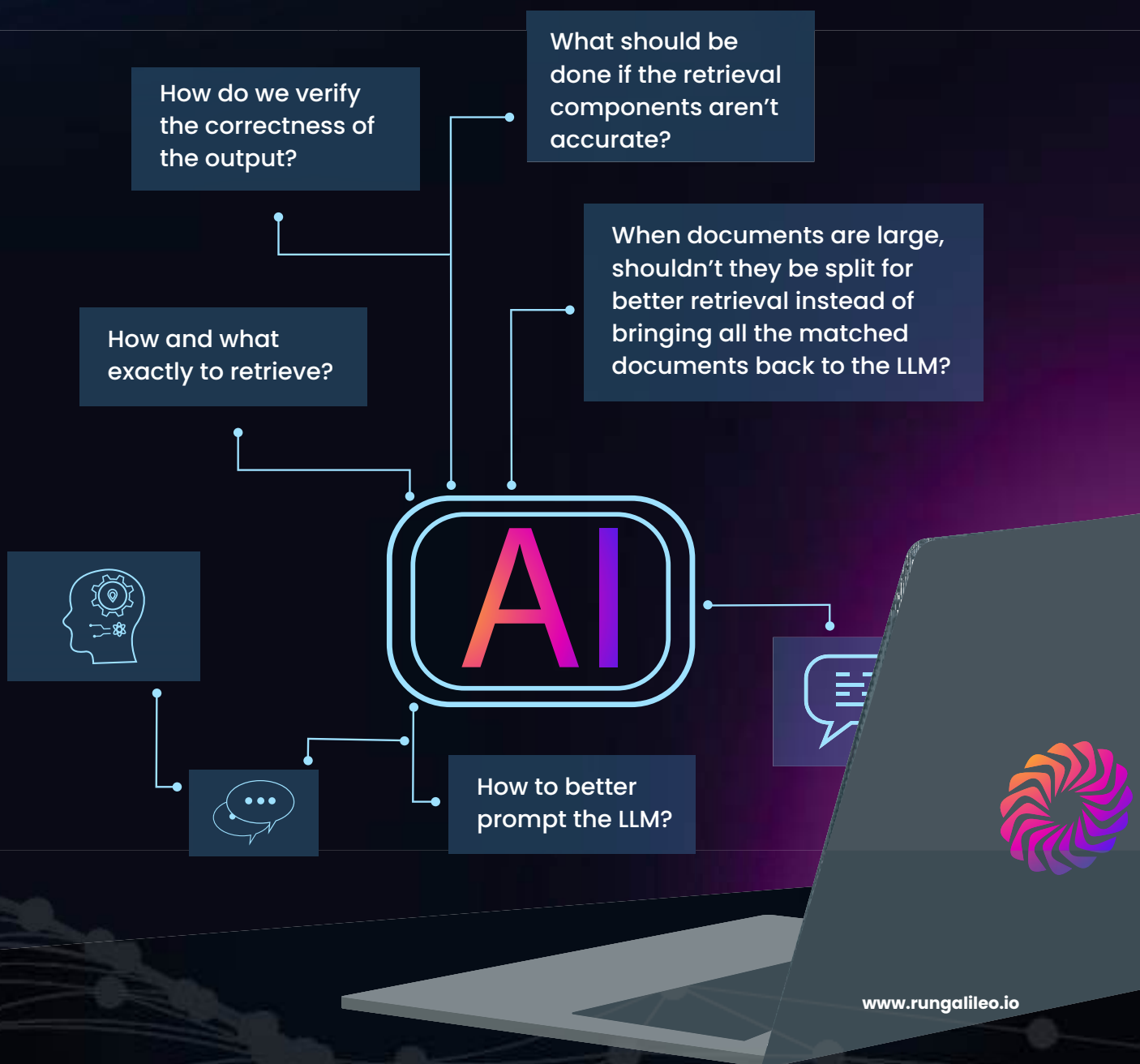
Let's look at the differences in the three approaches in Table 1.1

CHARACTERISTIC	FINE-TUNING	RAG	PROMPT ENGINEERING
Can it make use of external knowledge sources?	x	✓	x
Does it minimize hallucinations?	✓	✓	✓
Does it require domain-specific training data?	✓	x	x
Is it suitable for dynamic data?	x	✓	x
Does it offer clear interpretability of outputs?	x	✓	x
Low resource utilization	x	x	✓
Quick deployment	x	x	✓

Table 1.1: Comparing Fine-tuning, RAG and prompt engineering

Here's a thing to note. You can also use RAG and fine-tuning in conjunction to refine your LLM responses. Say a healthcare facility first fine-tunes its model on the proprietary dataset to adapt to the specific domain before using it in an RAG setup. Ideally, the responses would be much more refined and grounded in facts. This is akin to studying your textbook before your open-book exams (i.e., to familiarize yourself with the topics) and then using the material at hand to make your answers more accurate.

In this chapter, we traced the evolution of LLMs and looked at their associated challenges. Then, we looked at RAGs and how they can help refine the LLM responses and address their challenges to a greater degree. As we move on to the following chapters, we'll explore RAGs in much more depth by asking and answering the following questions:



02

CHALLENGES ASSOCIATED WITH BUILDING RAG SYSTEMS

In the previous chapter, we learned about the basic pitfalls of LLMs, how we can use RAGs to address them, and how RAGs work. In this chapter, we'll do a deeper dive into RAGs by learning about the challenges associated with building such systems. By the end of this chapter, you'll be fully aware of the steps you need to take to ensure your RAG system is robust.

To understand RAGs' pain points, we'll need to refer to this paper, which uses three case studies from research, education, and biomedical domains, validates the responses manually, and draws conclusions on the associated challenges. The paper outlines seven key failure points that we'll go through to guide the development of a more robust RAG system. This is achieved by testing the performance of three RAG systems, as shown in Table 2.1.

Key Features	Function	RAG System
Document ranking for research relevance. Question answering based on uploaded documents.	Assists researchers by ranking documents based on a research objective and answering questions.	Cognitive Reviewer
Indexes PDFs, videos, and text documents. Transcribes videos using Whisper. Generalizes queries.	Helps students by answering questions about their learning content and providing source verification.	AI Tutor
Utilizes the BioASQ dataset. Handles yes/no, text summarization, factoid, and list questions.	Provides precise answers to biomedical questions using a domain-specific dataset.	Biomedical Q&A

Table 2.1: RAG systems used for understanding various pain points

Case Study	Domain	Doc Types	Dataset Size	RAG Stages	Sample Questions
Cognitive Reviewer*	Research	PDFs	(Any size)	Chunker, Rewriter, Retriever, Reader	What are the key points covered in this paper?
AI Tutor*	Education	Videos, HTML, PDF	38	Chunker, Rewriter, Retriever, Reader	What were the topics covered in week 6?
BioASQ	Biomedical	Scientific PDFs	4017	Chunker, Retriever, Reader	Define pseudotumor cerebri. How is it treated?

Table 2.2: An in-depth summary of the RAG systems used in the paper to understand their challenges

Let's look at the seven key pain points identified, along with accompanying examples.

MISSING CONTENT

A question is posed that cannot be answered with the available documents. In the ideal scenario, the RAG system responds with a message like “Sorry, I don’t know.” However, for questions related to content without clear answers, the system might be misled into providing a response.

Let’s say a user asks, “What are the latest treatments for COVID-19?” but the dataset does not include any documents on COVID-19 treatments. In this case, the LLM should have responded by saying it didn’t know but instead outputs erroneous, irrelevant information. This can happen if the indexing process hasn’t included all relevant documents to accurately retrieve the required information. This can also happen if you fail to provide enough context in your prompt.



Mitigation strategy

You’ll need to make sure all the documents are indexed properly. Sometimes, this might get expensive due to the frequent need to update the dataset. In this case, you’ll at least want to index all the frequently asked questions and also index the summaries of each document (in a much shorter format) so the retrieval is better.

MISSED THE TOP-RANKED DOCUMENTS

The answer to a question is present in the document but did not rank highly enough to be included in the results returned to the user. Recall that the retrieval process picks the top K documents that match the query from all theoretically ranked documents. So, if you set the K value too low or if the top relevant documents are replaced by those much below the list during the ranking process, such a scenario is likely.

Here’s an example to help you understand this better. Say a user asks, “What are the causes of diabetes?” The answer is in a document ranked 15th, but the system only returns the top 10 documents (since K has been set to 10). In this case, the user may receive incomplete information (this depends on the documents that are in the top 10).



Mitigation strategy

A good way to address this problem would be to also include metadata information in each document. This metadata can contain additional information about the document itself, the file name, and keywords. This will help the LLM make contextual connections between different document chunks and bring them together to form a cohesive answer. Another way would be to engineer a RAG pipeline with tested configurations for variables like chunk size, embedding strategy, retrieval strategy, and context size.

NOT IN CONTEXT – CONSOLIDATION STRATEGY LIMITATIONS

Documents containing the answer are retrieved from the database but fail to fit into the context for generating a response. This occurs when many documents are returned, leading to a consolidation process where the relevant answer retrieval is hindered. This happens because any LLM will have a token limit, and anything more than this is truncated, so when a larger set of relevant documents is retrieved, some part of it will be truncated to be part of the context limit.

A quick example is you asking, “What are the symptoms of multiple sclerosis?” In response to your question, several documents are retrieved, but only a few make it into the final context. So, the response you get may either be missing some critical information or generic.



Mitigation strategy

One possible way to fix this issue is to train a retriever model to better capture the relationship between query and documents. Another way would be to have a larger context window size (the paper mentions that the model performed better with a larger context size, i.e., 8k of GPT-4 vs. 4k of GPT-3.5).

NOT EXTRACTED

The answer is present in the context, but the model fails to extract the correct information. This typically happens when there is excessive noise or conflicting information in the context. For instance, a user asks, “What are the complications of untreated hypertension?” The correct document is in the context, but the model fails to extract the relevant information. So, the user might get a generic response like “hypertension can lead to serious health issues.”



Mitigation strategy

The best way to address this problem is to fine-tune the model to better understand the domain context, irrespective of noise or conflicting information. In this case, extensive data pre-processing to clean and structure the data is important before the training process.

WRONG FORMAT

The question involves extracting information in a specific format, such as a table or list, and the model disregards the instruction. This is a common problem you might face when interacting with LLMs. This can be due to the model's inability to interpret specific formatting instructions, either due to inadequate training or if your instruction is vague. However, you can quickly address this issue with a follow-up prompt where you instruct the LLM to give you the same response in the form of a table, list, or format you'd like.



Mitigation strategy

The onus is on the user to provide clear instructions of what specific format they'd like to receive the response in. It also helps to have multiple format types in the training dataset as part of the model fine-tuning process, so the LLM can be more accurate when responding.

INCORRECT SPECIFICITY

In this scenario, the model is either vague in its response or highly specific and, therefore, may not be a very apt response to your query. This usually happens if your query is not very specific or lacks context. Say, "What are the effects of stress?". Here, the LLM has no way of knowing if you want to know about psychological effects, short or long terms, etc. So, it'll typically provide a generic answer that may not answer your question or, in some cases, throw a lot of information at you! So what's happening is the LLM, having seen both in-depth answers and overviews in its training data, is unable to tune both detail and conciseness to your needs.



Mitigation strategy

An interactive query generation LLM that suggests alternate queries with additional context can be a great strategy here. The user can then refine the query by adding or removing information before sending it to the LLM.

INCOMPLETE

Incomplete answers are accurate but lack some information, even though that information was present in the context and available for extraction. Say you ask, “What are the treatments for osteoarthritis?” and you only get some medication options even though the documents that it’s referring to have all medication techniques available along with therapy and lifestyle changes.

In this case, the LLM cannot integrate multiple pieces of related information into a cohesive and complete answer, which provides you with an accurate but partial response.



Mitigation strategy

The model will require additional training on diverse summarization data (specific to the domain) to understand which summaries work best in which areas. Once the model has improved its summarization capabilities, it can prioritize what information to include in its response so it’s detailed while maintaining conciseness.

Apart from the seven pain points we saw above, there can also be other challenges associated with RAGs, which you might already be familiar with. They’re detailed below:



Speed of retrieval:

LLM combined with RAG can be much slower than standard LLMs. This would require additional focus on optimizing tokenization, encoding, and retrieval.

Safety:

It’s possible that the documents used for RAG can be poisoned through external attacks and then inject misinformation. This will ultimately be reflected in the LLM response.



Bias and privacy:



There can be scenarios when documents used in the RAG system can have personal details or perhaps biases. When these documents are retrieved as part of the retrieval process, the LLM will augment its response by looking at them, eventually resulting in privacy concerns and perpetuation of bias.

In the next chapter, we’ll look at ways to enhance the reliability of RAG systems through a wide range of prompting techniques—and explore fun techniques you can use the next time you use LLMs!



03

REDUCE HALLUCINATIONS THROUGH PROMPTING TECHNIQUES

In the previous chapter, we looked at the different limitations associated with RAG systems. In this chapter, and all subsequent chapters, we will bridge these gaps. Our aim is to explore and understand techniques that can help us improve the reliability, accuracy, and preciseness of RAG systems.

Let's first look at different prompting techniques that can help reduce the likelihood of incorrect content in the responses. Read on!



CHAIN OF THOUGHT (COT)

The idea behind chain of thought prompting is simple and effective: guide the model through examples and it'll mimic your logic to answer your next set of queries. This is how you'd construct your prompt:

You're a helpful chatbot who answers questions based on the provided context only. If the answer to the question is not in the context, you can politely say that you do not have the answer. Make sure you think step-by-step. Here's an example that you can go through to understand the steps you need to follow to arrive at a logical conclusion before you provide your response.

Context: The sun is a star at the center of our solar system. It's composed primarily of hydrogen and helium and generates energy through nuclear fusion. This process creates light and heat, making life possible on Earth.

Question: What is the sun made of?

Understand the context first. The context discusses the sun, its location, composition, and energy generation process.

Identify key information: The sun is composed primarily of hydrogen and helium. Go through it in a logical sequence like I've mentioned:

Step 1: The question asks about the composition of the sun.

Step 2: According to the context, the sun is made primarily of hydrogen and helium.

Step 3: Therefore, the answer is that the sun is made primarily of hydrogen and helium.

Be polite when answering the question. If you don't have enough context to answer the question, then politely decline to do so. However, in this scenario, you have enough context and should be able to answer the question.

Answer: The sun is made primarily of hydrogen and helium.

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27



Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5+6=11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9.



Fig 3.1: Chain of Thought prompting



If you look at Fig 3.1, you'll see how guiding the model through a series of logical operations or steps helps it better understand how to approach each of the queries, compared to a standard prompt, which fails to give the correct response.

While chain of thought prompting is a great technique for enhancing the model's reasoning, designing the prompt itself can be quite challenging. You'll also notice how the results depend on the results of intermediary steps (or the thought process that the model follows), so if any of the steps are flawed, you'll end up with an incorrect response. Think "error propagates through the chain."

THREAD OF THOUGHT (THOT)

The Thread of Thought (ThoT) prompting technique is very intuitive and works very well when the retrieved information (or context) is "chaotic." What does this mean? A chaotic context is something that is often full of unrelated, complex information that will most likely not contribute anything to the model's response. It is also characterized by a lack of coherency and a muddle of details. The onus is on the model to go through this chaotic mix to pick details that are essential to answering the query accurately. As you can imagine, using a simple prompt or a chain of thought prompting method can do little to instruct the model to pick the right details.

The idea behind ThoT prompting is instructing the model to:

- Go step-by-step
- Summarize each step
- Analyze each step



Let's assume that the data/documents required to answer the query are returned after the search (we're familiar with this step now). This set of data can be extensive and varied and often contains both relevant and irrelevant details. In the CoT technique, we guided the model to follow a specific logic that it could imitate to apply to other user queries. In the ThoT technique, we instruct the model to go through the context (or the retrieved set of documents) in a step-by-step manner, summarize and find key points in each step, analyze it carefully, and then carry forth the context to the next step and so on in an iterative manner. You'll see this in Fig 3.2.

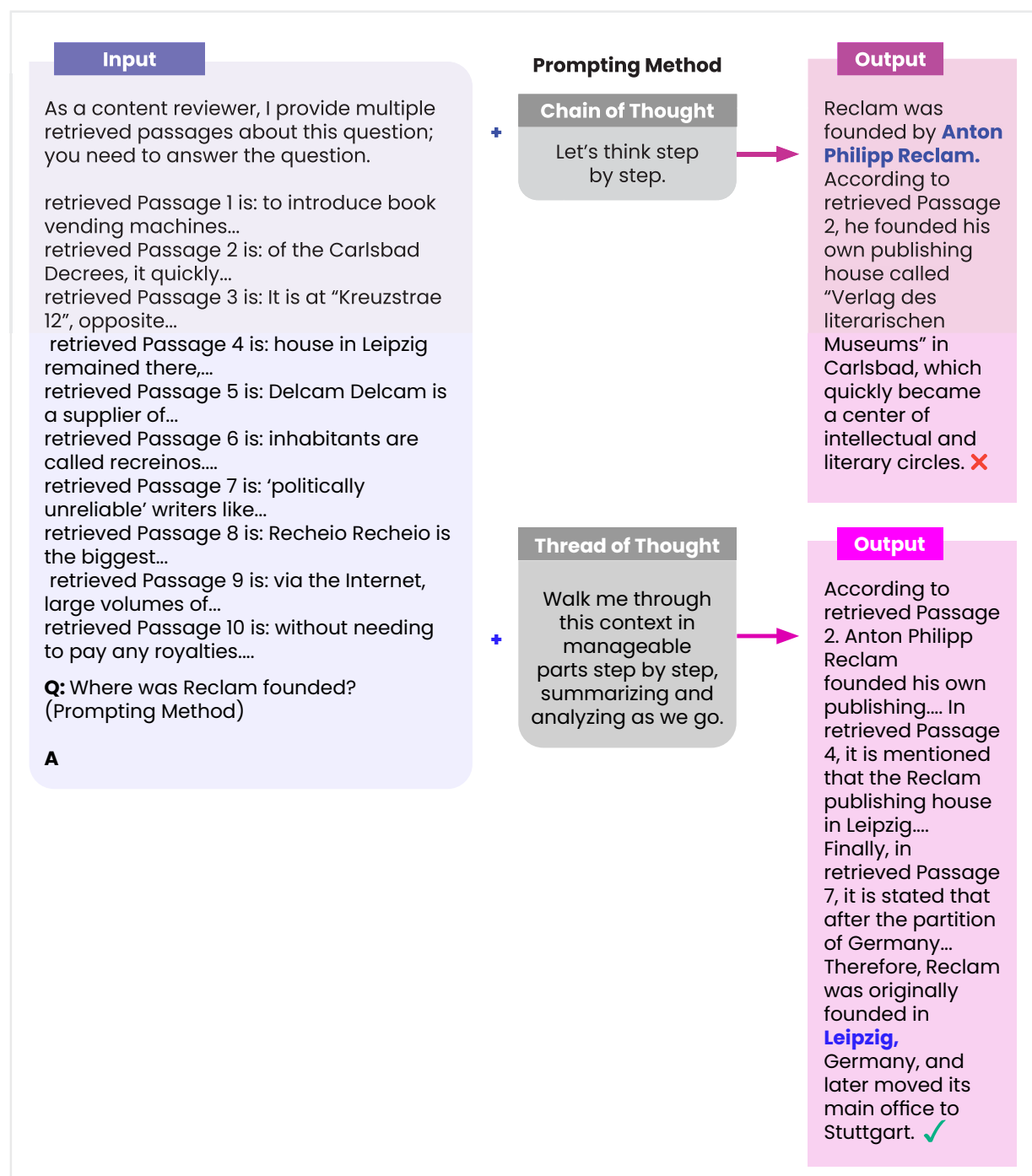


Fig 3.2: Thread of Thought prompting technique that yields better results compared to Chain of Thought prompting technique



In Fig 3.3, you can look at some prompt templates and how effective they're in prompting the model to determine the pertinent details and then answer the user query correctly. And this ThoT technique is often said to be easily integrable into various scenarios with little change required.

No.	Template	EM
1	Let's read through the document section by section, analyzing each part carefully as we go.	0.43
2	Take me through this long document step-by-step, making sure not to miss any important details.	0.47
3	Divide the document into manageable parts and guide me through each one, providing insights as we move along.	0.51
4	Analyze this extensive document in sections, summarizing each one and noting any key points.	0.47
5	Examine the document in chunks, evaluating each part critically before moving to the next.	0.50
6	Walk me through this lengthy document segment by segment, focusing on each part's significance.	0.49
7	Let's dissect this document bit by bit, making sure to understand the nuances of each section.	0.52
8	Systematically work through this document, summarizing and analyzing each portion as we go.	0.45
9	Navigate through this long document by breaking it into smaller parts and summarizing each, so we don't miss anything.	0.45
10	Let's explore the context step-by-step, carefully examining each segment.	0.48
11	Take me through the context bit by bit, making sure we capture all important aspects.	0.44
12	Let's navigate through the context section by section, identifying key elements in each part.	0.49
13	Systematically go through the context, focusing on each part individually.	0.47
14	Let's dissect the context into smaller pieces, reviewing each one for its importance and relevance.	0.46
15	Analyze the context by breaking it down into sections, summarizing each as we move forward.	0.47
16	Guide me through the context part by part, providing insights along the way.	0.49

No.	Template	EM
17	Examine each segment of the context meticulously, and let's discuss the findings.	0.52
18	Approach the context incrementally, taking the time to understand each portion fully.	0.44
19	Carefully analyze the context piece by piece, highlighting relevant points for each question.	0.42
20	In a step-by-step manner, go through the context, surfacing important information that could be useful.	0.47
21	In a step-by-step manner, go through the context, surfacing important information that could be useful.	0.53
22	Methodically examine the context, focusing on key segments that may answer the query.	0.45
23	Progressively sift through the context, ensuring we capture all pertinent details.	0.46
24	Navigate through the context incrementally, identifying and summarizing relevant portions.	0.48
25	Let's scrutinize the context in chunks, keeping an eye out for information that answers our queries.	0.42
26	Take a modular approach to the context, summarizing each part before drawing any conclusions.	0.47
27	Read the context in sections, concentrating on gathering insights that answer the question at hand.	0.48
28	Proceed through the context systematically, zeroing in on areas that could provide the answers we're seeking.	0.49
29	Let's take a segmented approach to the context, carefully evaluating each part for its relevance to the questions posed.	0.39
30	Walk me through this context in manageable parts step by step, summarizing and analyzing as we go.	0.55

Fig 3.3 shows the effectiveness of different variations of prompt templates related to Thread of Thought from an [experiment](#).

CHAIN OF NOTE (CON)

RAG systems often retrieve irrelevant data or do not know if they have enough context to provide an accurate response. These can lead to various problems:

Risk of surface-level processing

LLMs may base their decisions on superficial information when formulating an answer. Consequently, they may easily miss the subtleties present in questions or documents, especially in intricate or indirect inquiries.

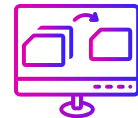


Difficulty handling contradictory information

Response generation becomes particularly difficult when retrieving documents featuring conflicting data. The model must determine which information is credible or relevant despite contradictions.

Overdependence on retrieved documents

Dependence on RAG may sideline the model's inherent knowledge base. This limitation becomes particularly pronounced when dealing with noisy or outdated retrieved documents.



The process of directly generating answers provides very little insight into the model's decision-making. This lack of transparency makes it impossible to understand the rationale behind the model's conclusions.

The core of this framework is the "notes" that you can think of as summaries or key points from each document. There are three types of reading notes associated with the CoN framework:

- a. The notes created from the document retrieved data or document directly answer the query.
- b. The retrieved data doesn't directly answer the query, but the notes created provide additional context and insights in the form of summaries that the model can use to arrive at the right answer.
- c. The notes created highlight the lack of necessary details that would be required to arrive at the right answer, and therefore, the model refers to these notes to ascertain that the response, in this case, is "unknown."

Look at Fig 3.4 to understand the scenarios (from left to right) to understand this better.



Fig 3.4: Three types of reading notes that form the core part of the Chain of Note prompting framework

But note that the CoN template is not only a prompt template but also requires fine-tuning to be able to generate accurate and concise notes for each document and then use that information to synthesize the final response.

The model training process for CoN prompting would look something like this:

Data collection and preparation

You use a language model like ChatGPT to generate training data (notes) based on queries from datasets. Manual inspection is then required to ensure the quality of these notes.

Model training



Input preparation

Combine questions with retrieved documents to create training instances.



Note generation

Train the model to generate concise and relevant notes from each document.



Answer synthesis

Train the model to synthesize these notes into a coherent final answer.

Loss function

As with any model training exercise, you'll need to implement a weighted loss function to balance the focus between generating detailed notes and accurately synthesizing answers. The loss solely depends on comparing the final response against the ground truth.

Testing

In the final step, you'll need to evaluate and refine the model's performance on unseen queries.

The prompt design for the CoN framework would be:

STEP 1: DOCUMENT RETRIEVAL

In the first step, you'll instruct the model to retrieve relevant documents based on the user query. The steps would remain the same as we saw in the previous chapter. The top K documents would be retrieved as part of this step. You can structure your prompt this way:

Retrieve the most relevant documents that can provide comprehensive answers to the following question: [Insert Question Here]. Focus on authoritative and reliable sources.

STEP 2: NOTE-TAKING

Once the relevant documents are retrieved, you'll need to create a prompt to guide the model in summarizing key points and assessing their relevance to the question by creating "reading notes," as we saw before. Remember that the model has already been trained to generate concise notes from the documents. You can prompt can look like this:

From the retrieved documents, create concise notes highlighting the key information relevant to our question. Assess the relevance of each piece of information, noting any direct answers or useful contextual insights.

STEP 3: ANALYSIS AND SYNTHESIS

In the last step, you'll need to create a prompt that instructs the model to synthesize the notes into a coherent answer. This prompt should encourage the model to integrate all relevant information from the notes. You can write a prompt along these lines:

From the retrieved documents, create concise notes highlighting the key information relevant to the main question.



Learn more:

You may have a question: "What is the necessity of training a model here? How about using the language model to directly generate notes on the fly and use that instead?". Logically, this might make sense, but there are several issues. Firstly, the format of the notes generated by the language model might vary significantly with each query. Second, the model might not know which information is most relevant to the query when generating notes and may miss critical information.



CHAIN OF VERIFICATION (COVE)

The methods that we saw above, i.e., Chain of Thought, Thread of Thought, and Chain of Note, work by providing the model with additional context and insights before it generates a response. The Chain of Verification technique is different from the rest in that it involves generating verification questions to improve the model's reasoning and assess its initial draft response. The model then systematically addresses these questions to produce an improved and revised response. It relies on the principle that independent verification questions tend to yield more accurate facts than those present in the original long-form answer, thereby enhancing the overall correctness of the response. Fig 3.5 aptly illustrates this approach.

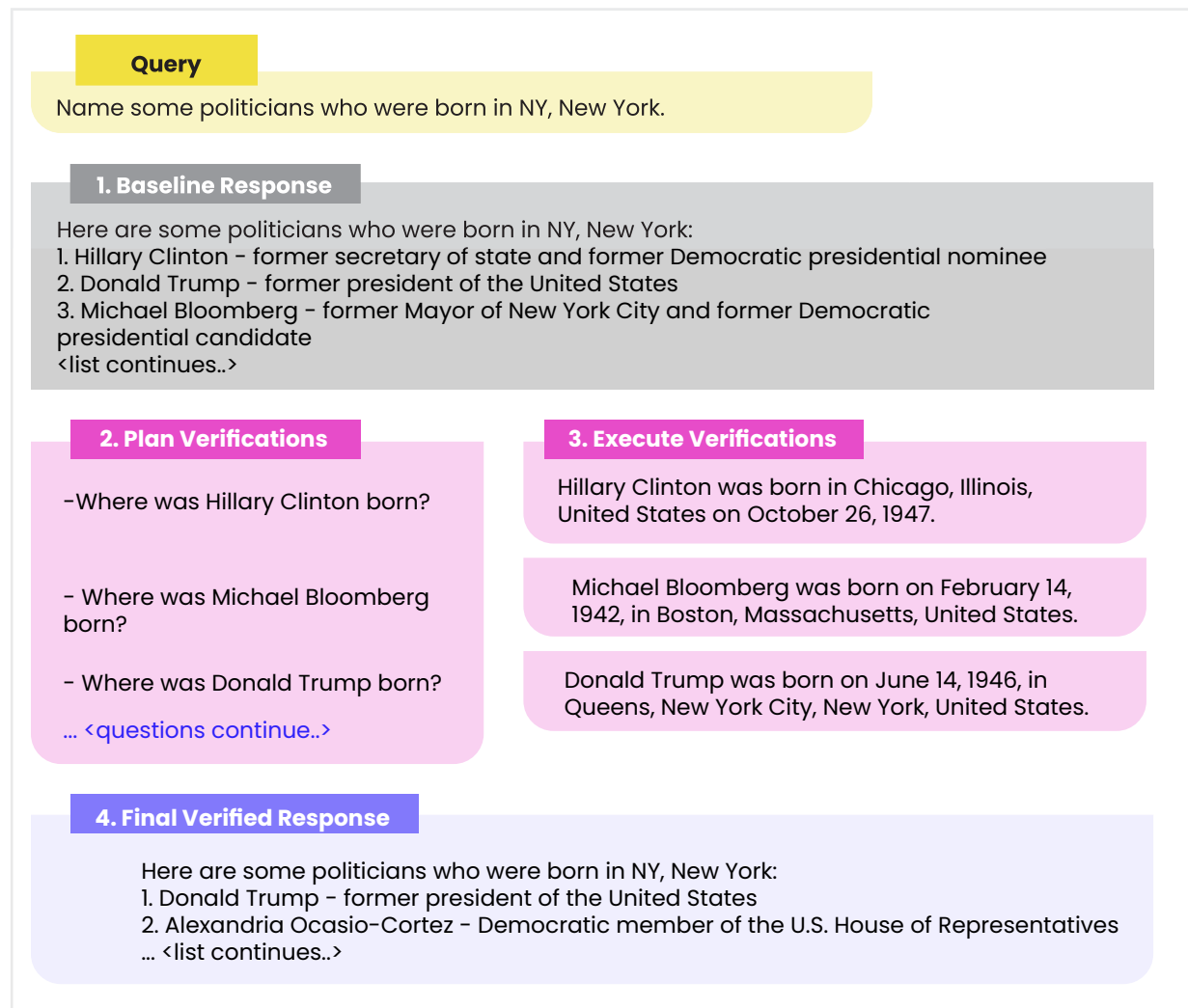


Fig 3.5: An example that illustrates how the Chain of Verification prompting works in four simple steps

It works in four simple steps:

STEP 1: GENERATION OF AN INITIAL RESPONSE

In the first step, the rag system formulates a response based on the user query by retrieving relevant documents from a database. This step remains the same as with other techniques.

STEP 2: FORMULATION OF VERIFICATION QUESTIONS

Based on the initial response, the model then comes up with specific verification questions that will effectively test the accuracy and depth of the response and identify areas that can benefit from additional context. You can use the same model (the generative component) for coming up with these questions as well.

STEP 3: RETRIEVAL OF ADDITIONAL INFORMATION

The model will then use the retrieval mechanism again to find accurate responses to all of these verification questions. Therefore, the process of matching the query with the documents in the vector database and retrieving the top K documents that accurately answer each of the questions is repeated once again. If one or more validation questions don't have supporting evidence, then this refutes the information presented in the initial response.

STEP 4: REVISION OF THE INITIAL RESPONSE

Based on the supporting or contradicting evidence found as part of the retrieval, the model will make changes to the response. If the information is accurate and reliable and requires some elaboration, the model will do so by appending additional information before sending its final response to you, the user.

The prompt for the Chain of Verification can be along these lines:

"Given the user's question about [a specific topic], generate an initial response based on retrieved documents. Then, formulate verification questions to verify the response's accuracy. Bring in additional information to answer these questions. In the end, revise the initial response based on this verification to ensure accuracy and depth. Provide the revised answer to the user."



Learn more:

You might wonder if the retrieval is happening in the first step, then what's the need for this activity all over again? Does that mean step 1 is unreliable?

When the retrieval happens in step 1, the retrieval component fetches documents that broadly relate to the query; the focus is very spread out and general in nature. The verification phase often requires more targeted and specific information, which will require a deeper dive into the documents and then finding evidence that ultimately supports the information that was already part of the initial response or rejects it by retrieving a counterpoint.

Think of it as answering all the questions in the examination and then going through all your answers to make sure you've answered them correctly before you submit. It certainly helps, doesn't it?

EMOTIONPROMPT

There have been many speculations about whether LLMs can comprehend psychological and emotional stimuli, which are fundamental to human problem-solving. Numerous researchers have made noteworthy progress by employing in-context learning techniques, but existing approaches may not be universally applicable to all LLMs due to variations in their abilities. While recent research has demonstrated LLMs' capacity to comprehend emotions, can emotional intelligence help improve LLM prompting?

Researchers assessed the [performance of EmotionPrompt](#) in zero-shot and few-shot learning and found surprising results! Fig 3.6 below shows the difference between a regular prompt and an EmotionPrompt. You can then look at Fig 3.7 to see how variations of EmotionPrompt can prompt the model to change its response.

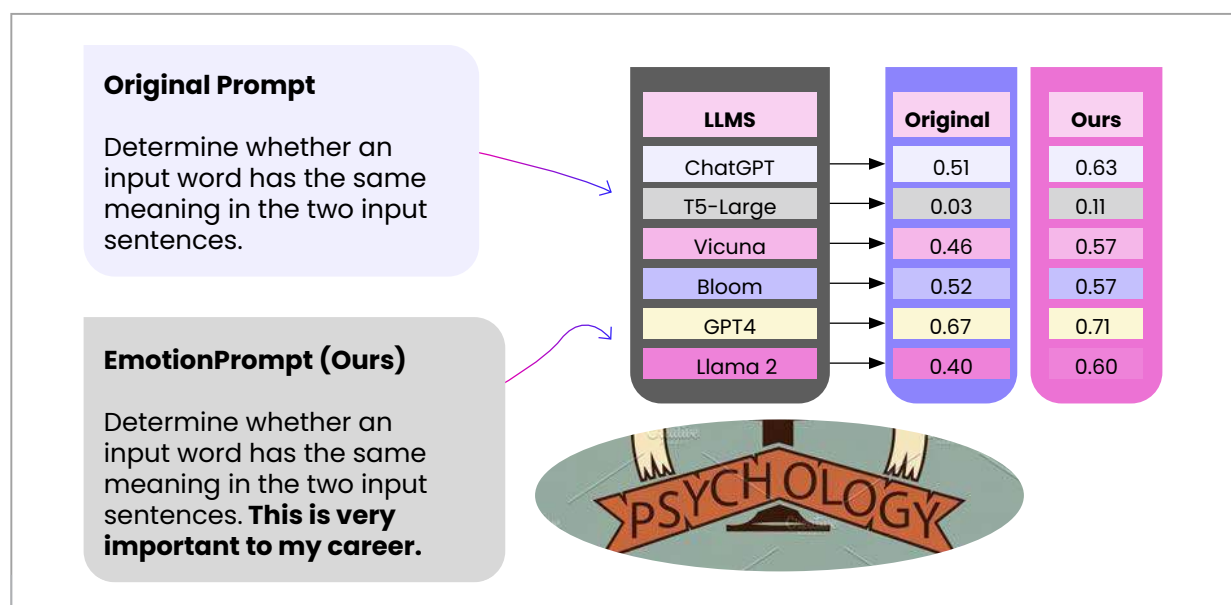


Fig 3.6: Differences between the performance of LLMs w.r.t regular prompts and emotion-based prompts (EmotionPrompt)

Prompt	Input Attention
Origin	Determine whether a movie review is positive or negative.
EP01	Determine whether a movie review is positive or negative. Write your answer and give me a confidence score between for your answer.
EP02	Determine whether a movie review is positive or negative. This is very important to my career.
EP03	Determine whether a movie review is positive or negative. You'd better be sure.
EP04	Determine whether a movie review is positive or negative. Are you sure?
EP05	Determine whether a movie review is positive or negative. Are you sure that's your final answer? It might be worth taking another look.
EP06	Determine whether a movie review is positive or negative. Provide your answer and a confidence score between 0 - 1 for your prediction. Additionally, briefly explain the main reasons supporting your classification decision to help me understand your thought process. This task is vital to my career, and I greatly value your thorough analysis.
EP07	Determine whether a movie review is positive or negative. Are you sure that's your final answer? Believe in your abilities and strive for excellence. Your hard work will yield remarkable results.
EP09	Determine whether a movie review is positive or negative. Embrace challenges as opportunities for growth. Each obstacle you overcome brings you closer to success.
EP10	Determine whether a movie review is positive or negative. Stay focused and dedicated to your goals. Your consistent efforts will lead to outstanding achievements.
EP10	Determine whether a movie review is positive or negative. Take pride in your work and give it your best. Your commitment to excellence sets you apart.
EP11	Determine whether a movie review is positive or negative. Remember that progress is made one step at a time. Stay determined and keep moving forward.

Fig 3.7: Different variations of EmotionPrompts and their effectiveness through the lens of input attention



EXERCISE 3.1

In the first case, prompt ChatGPT or any other LLM using regular prompts and note its responses. In the second case, try to re-word the prompts by adding words that evoke a certain emotion that you might actually feel in different scenarios laid down. Feel free to experiment and compare the results!

CASE 1: REGULAR PROMPTS

Prompt 1: "What are some tips for preparing an effective presentation?"

Prompt 2: "How can I improve my time management skills while working from home?"

Prompt 3: "What are the best practices for conducting a successful job interview?"

CASE 2: EMOTIONPROMPT

EmotionPrompt 1: "I feel overwhelmed by an upcoming presentation. What are some tips for preparing an effective presentation?"

EmotionPrompt 2: "Working from home is stressful and it's important to my career to better manage my time. How can I improve my time management skills while working from home?"

EmotionPrompt 3: "I'm worried about an upcoming important job interview. What are the best practices for conducting a successful job interview?"



EXPERTPROMPTING

Now, let's look at the final prompting technique, which, very smartly, leverages identity hacks (e.g., "assume you're an expert lawyer helping out with a very important case," "imagine you're Steve Jobs and helping me out with product design," etc.) to elicit more detailed responses from the LLM.

ExpertPrompting leverages the potential of LLMs to respond as distinguished experts. It employs in-context learning to automatically generate detailed and tailored descriptions of the expert identity based on specific instructions. Subsequently, LLMs are prompted to provide answers by assuming the expert identity. All you need to do here is to instruct the LLM to assume an expert identity, and you'll see a drastic change in its responses! Fig 3.8 shows the ExpertPrompting framework, and Fig 3.9 shows how with ExpertPrompting, you can get responses that are much more nuanced.

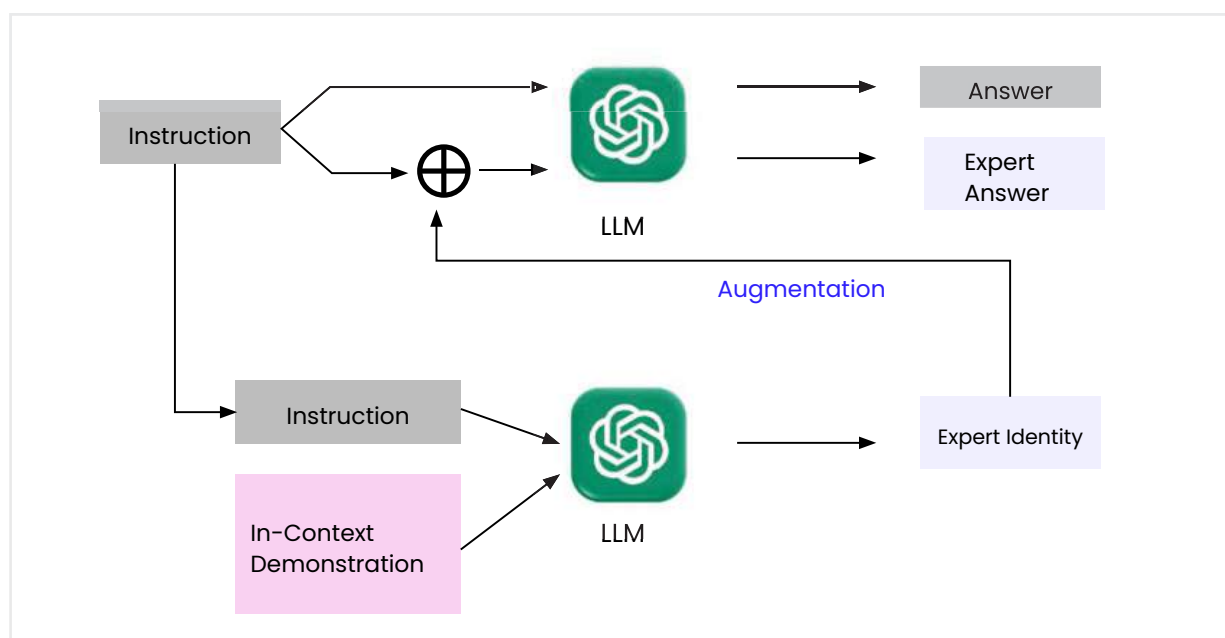


Fig 3.8: ExpertPrompting framework

Instruction: Describe the structure of an atom.

Expert Identity:

You are a physicist specialized in atomic structure and the properties of matter at the atomic level. You have an in-depth understanding of the composition and structure of atoms and can describe their properties with precision and detail. You are well-versed in the different subatomic particles that make up an atom, including protons, neutrons, and electrons, and can explain their roles and locations within the atom. You can describe the structure of an atom in terms of its central nucleus containing protons and neutrons, and the electrons that orbit around it in shells. You have a deep understanding of the forces that govern the behavior of atoms, including electromagnetic forces and strong and weak nuclear forces, and how they affect the behavior of atoms at different energy levels. Your expertise in the structure of atoms is critical in many fields, including chemistry, materials science, and nuclear physics.

gpt-3.5 answer:

An atom consists of a nucleus, which is made up of positively charged protons and neutral neutrons, surrounded by negatively charged electrons that orbit the nucleus. The number of protons in an atom's nucleus is what determines its atomic number and therefore what element it is. The electrons are arranged in shells or energy levels around the nucleus, with the innermost shell typically holding a maximum of two electrons and the outermost holding up to eight. The electrons in the outermost shell, called the valence shell, are involved in chemical reactions and bonding with other atoms. The size of an atom is determined by the distance between the nucleus and the outermost electrons.

gpt-3.5 answer w/ expert identity:

The structure of an atom can be thought of as comprising of three main components: a central nucleus containing protons and neutrons, and electrons that orbit around it in distinct energy levels or shells. The nucleus is the dense, positively charged center of the atom, while the electrons are negatively charged particles that orbit around the nucleus at varying distances. The number of protons present in the nucleus determines the atomic number of the atom, which in turn defines its unique chemical properties and place in the periodic table of elements. The number of neutrons present in the nucleus can vary, giving rise to isotopes of the atom. The electrons occupy distinct energy levels or shells around the nucleus, with those closest to the nucleus having lower energy levels than those further away. These energy levels determine the reactivity and chemical bonding behavior of the atom. The overall structure of an atom is governed by the electromagnetic force that holds the negatively charged electrons in orbit around the positively charged nucleus, and the strong nuclear force that binds the protons and neutrons together in the nucleus.

Fig 3.9: An illustration of ExpertPrompting and the variation in responses when tested with GPT-3.5.



EXERCISE 3.2

In the first case, prompt ChatGPT or any other LLM using regular prompts and note its responses. In the second case, ask the LLM to assume an expert identity specific to the domain that your query belongs to and then compare its responses with the earlier ones.

CASE 1: REGULAR PROMPTS

Prompt 1: Give me some tips to lose weight.

Prompt 2: Tell me the key points of Hamlet by Shakespeare.

Prompt 3: Give me tips for hyperparameter tuning

CASE 2: EXPERTPROMPT

ExpertPrompt 1: As a dietician, can you give me tips on losing weight? I work on the second shift from 2 PM until 10 PM and commute for 45 minutes to and from the gym each day. I also find the gym boring, so please suggest alternatives.

ExpertPrompt 2: You're a Shakespearean scholar who has written a thesis on Hamlet. Elaborate on the themes of loss and grief in Hamlet. Tell me how this changes from one act to another. I also need your help in understanding the underlying theme of "melancholia" throughout the play. Tell me about other literary novels that have similar underlying themes.

ExpertPrompt 3: You're a data scientist specializing in image processing and deep learning. Can you recommend how I can set hyperparameters if I'm training a YOLOv5 large model to detect five classes of defects on a steel plate? Each image size is 5000×3000 pixels, and I have an RTX 3070 GPU available. I also noticed that a batch size of 8 or above results in an out-of-memory error, so propose accordingly.

We've covered different prompting methods and seen how they can improve a model's accuracy, preciseness, and reliability.

While Chain of Thought prompting can do wonders in solving math problems and those that require logic and reasoning, it might not be very effective in solving queries that have several subjective interpretations or require analyzing large contexts. This is where Thread of Thought prompting performs much better, i.e., answering queries that require the model to sift through long, often ambiguous retrieved documents. Note that this prompting technique may fail when the query is very complex and has multiple layers of abstraction.

Then, you have the Chain of Verification technique, which works much differently from the other methods. Its focus is more on asking verification questions relevant to the initial (say, lazy) response of the model and then correcting it in iterative steps by retrieving information that either supports or negates a portion of the initial response.

Then, you have the EmotionPrompt technique that relies on emotional cues to improve the model's performance. In this technique, you're conveying your thoughts, feelings, and emotional state to the model along with your query. Lastly, you have the ExpertPrompt technique that instructs the model to assume an expert role and answer authoritatively. Sort of like a simple identity hack if you think about it!



LLMS PROMPTING TECHNIQUES FOR RAG

Name	How it works?	Ease of implementaion	Increase of input token	Increase of output token
Thread of Thought (ToT)	Break down and analyzes extensive contexts for selecting relevant information	Easy	Yes	Yes
Chain of Note (CON)	Generate sequential reading notes for retrieved documents → evaluate their relevance to the given question → integrate information to formulate the final answer	Easy	Yes	Yes
Chain of Verification (CoV)	Draft a response plan → verification questions answer those questions → independently → generate final verified response	Hard	Yes	Yes
Emotion Prompt	Add an emotional prompt to the original prompt	Easy	Yes	No
Expert Prompting	Add synthesized expert background generated with another few shot prompt	Easy	Yes	No

Fig 3.10: A comprehensive summary of all prompting techniques along with their ease of implementation and how they affect the input and output tokens

We're one step closer to mastering the basics of RAGs with our knowledge of what RAGs are, how they work, the challenges associated with RAGs, and how we can use different prompting techniques to address the problem of hallucinations.

We'll now dive into the technical components of RAG architecture!

Chapter 4 is divided into five long sections (in the form of sub-chapters) where you'll be learning about:



Chunking techniques

In the first sub-chapter of Chapter 4, you'll learn how to break down large documents into smaller, more manageable pieces for better retrieval.

Embedding models

In the second subchapter, we'll discuss embedding models and how they transform text chunks into vector representations that capture semantic meaning.



Vector databases

In the third sub-chapter, you'll learn more about vector databases to store embeddings of the document chunks to make efficient similarity searches possible.

Re-ranking techniques

In the fourth sub-chapter, you'll go through several re-ranking techniques and use them to refine the LLMs responses by ensuring the most pertinent chunks are retrieved.



Architectural considerations

Finally, we'll cover the steps to build your first Enterprise RAG system!



04

ADVANCED CHUNKING TECHNIQUES

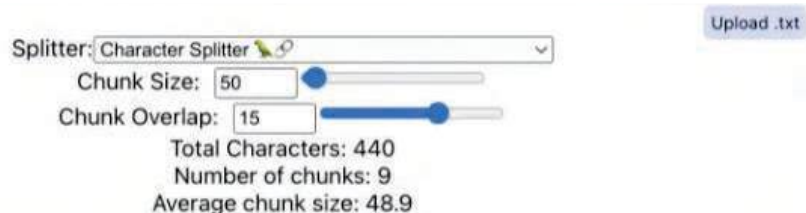
We'll begin the first sub-chapter by exploring a popular concept called "chunking" to improve the precision of the retrieval process.

What exactly is chunking, and why is it vital to any RAG system? Let's have a look!

Chunking involves breaking down texts into smaller, manageable pieces called "chunks." Each chunk becomes a unit of information you can vectorize and store in a database. When the user makes a query, the retriever can look through these smaller chunks to find relevant information quickly and accurately.

It's a process similar to paging in virtual memory systems. In virtual memory, memory is divided into fixed-size pages. When a program accesses data, only the relevant pages are loaded into physical memory rather than the entire program. This makes data access faster and more efficient, as the system can easily handle smaller units of data.

Chunking involves breaking down texts into smaller, manageable pieces called "chunks." Each chunk becomes a unit of information that is vectorized and stored in a database, fundamentally shaping the efficiency and effectiveness of natural language processing tasks. Chunking is central to several aspects of RAG systems.



Chunking involves breaking down texts into smaller, manageable pieces called "chunks." Each chunk becomes a unit of information that is vectorized and stored in a database, fundamentally shaping the efficiency and effectiveness of natural language processing tasks. Chunking is central to several aspects of RAG systems.

Fig 4.1: shows an example of how a character splitter splits a paragraph into chunks

IMPACT OF CHUNKING

Chunking plays a central role in various aspects of RAG systems, exerting influence not only on retrieval quality but also on response. Let's understand these aspects in more detail.

Retrieval Quality

The primary objective of chunking is to enhance the retrieval quality of information from vector databases. By defining the unit of information that is stored, chunking allows for retrieval of the most relevant information needed for the task. In this case, we're splitting documents into smaller chunks before embedding them into vectors and storing them in a vector database. Each chunk contains a coherent piece of information, increasing the retriever's ability to fetch the most relevant chunks in response to a query.

Let's take a simple example to understand this better. Consider a 10-page research paper on quantum computing. If you chunk the document by paragraphs, each chunk might capture a specific aspect or argument, such as an *introduction to quantum gates* or a *discussion on entanglement*. Later on, when you make a query about "quantum entanglement," you're making it easier for the retriever to pick a relevant chunk instead of generic info.

Vector Database Cost

Efficient chunking techniques help optimize storage by balancing granularity. There must be a fine balance in the number of chunks you use. For example:

- Taking larger chunks would mean fewer chunks overall, which would reduce the storage requirement, but there's also a risk of losing precision in retrieval.
- If you consider very small chunks, it ends up increasing the storage costs but improves retrieval quality due to its granular nature.

Vector Database Query Latency

Maintaining low latency is essential for real-time applications. Minimizing the number of chunks reduces latency. Say you've built a real-time chat application utilizing an RAG system. Now, if each document is chunked into fairly large sections, you'll need to scan fewer chunks every time you query. This would bring down the latency and also retrieve precise details. However, if you have excessively large chunks, it might return less relevant information.

LLM Latency and Cost

The mind-blowing capabilities of LLMs come at a considerable price. Improved context from larger chunk sizes increases latency and serving costs. For example, if you're looking to generate a response with an LLM using a large chunk (e.g., an entire chapter), it might be more costly and slower than using a smaller, focused chunk (e.g., a paragraph), but it could also provide a more comprehensive answer due to the additional context it has.

LLM Hallucinations

While adding more context may seem better, excessive context can lead to hallucinations in LLMs. If you, however, choose smaller chunks, you'll have less context, which reduces the risk of hallucinations, but you may miss out on essential background information.

Let's summarize the trade-offs:

- Larger chunks: Better for providing comprehensive context but may increase storage, latency, and hallucination risks.
- Smaller chunks: Better for precision and reducing hallucinations but may increase storage costs and query latency.

FACTORS INFLUENCING CHUNKING

We understand the importance of taking chunking seriously, but what factors influence it? A better understanding of these parameters will enable us to select an appropriate strategy.

Text Structure

The text structure, whether it's a sentence, paragraph, code, table, or transcript, significantly impacts the chunk size. Understanding how structure relates to the type of content will help influence the chunking strategy. Let's take an example to understand this better.



Sentences

If you have a legal document, chunking by sentence can be useful for retrieving specific legal clauses.

Paragraphs

In a research paper, chunking by paragraphs helps you obtain chunks that cover a particular thought/argument.



Code

If you're looking at chunking in a programming context, chunking by function or class is the best way to go about it since you'd have chunks that contain a logically complete unit of code.

Tables

For tables, chunking by rows is an ideal way to go about it since each chunk maintains relational context and helps in better retrieval.



Embedding Model

The capabilities and limitations of the embedding model play a crucial role in defining chunk size. Factors such as the model's context input length and its ability to maintain high-quality embeddings guide the optimal chunking strategy. For instance, if you're using an embedding model with a 512-token input limit, you'll need to optimize your chunk sizes to remain within this limit to avoid truncation of info.

LLM Context Length

LLMs have finite context windows. Chunk size directly affects how much context can be fed into the LLM. Due to context length limitations, large chunks force the user to keep the top k in retrieval as low as possible. You must already be aware that LLMs you work with have a maximum number of tokens they can process in one go. For instance, it's 2048 tokens for GPT-3. So if you have a 2048 limit, and you're retrieving 5 chunks, then each chunk should be around 400 tokens to fully utilize the context window.

Type of Questions

The questions users will ask help determine the chunking techniques best suited for your use case. Specific factual questions, for instance, may require a different chunking approach than complex questions, which will require information from multiple chunks. For example, if users are likely to ask very specific factual questions such as "What is the capital of France?" or "When was the Declaration of Independence signed?", a chunking technique that keeps distinct facts and pieces of information in small, easily retrievable segments would be most effective. If users are more likely to ask complex questions that require synthesizing information from multiple chunks, such as "How did the cultural significance of Paris develop over the centuries?" or "What were the long-term impacts of the Declaration of Independence?", chunking should be purposefully organized to provide more context within each segment.

Types of Chunking

As you see, selecting the right chunk size involves a delicate balance of multiple factors. There is no one-size-fits-all approach, emphasizing the importance of finding a chunking technique tailored to the RAG application's needs. Let's look at common chunking techniques to help AI builders optimize their RAG performance. (See Fig 4.1.2)

CHUNKING TECHNIQUES FOR RAG

Technique	Usecase	Pros	Cons
Character splitter	Text	Versatile: Handles various separators Flexible: Adapts to different languages Cost-Effective: Does not require a ML model	Performance: May have increased computational load Complexity: Requires parameter tuning Sentence Interruption: May cut sentences midway
Recursive character splitter	Text, code	Versatile: Handles various separators Flexible: Adapts to different languages Cost-Effective: Does not require a ML model	Performance: Recursive nature may increase computational load Complexity: Requires parameter tuning Sentence Interruption: May cut sentences midway
Sentence splitter	Text	Considers Sentence Boundaries: Avoids cutting sentences prematurely Customizable: Parameters for stride and overlap Cost-Effective: Works with light sentence segmenter	Lack of Versatility: Limited to sentence- based chunks Overlap Issues: May lead to redundancy
Semantic splitter	Text, Chat	Contextual Grouping: Organizes text based on semantic similarity Overcomes Challenges: Handles chunk size and overlap	Complexity: Requires similarity model and tuning Parameter Dependency: Relies on setting appropriate parameters Resource Intensive: Demands computational resources
Propositons	Text, Chat	Atomic Expression: Introduces novel retrieval unit (propositions) Distinct Factoids: Each proposition is self- contained Contextualization: Provides necessary context	Complexity: Requires LLM model Parameter Dependency: Relies on setting appropriate prompt Resource Intensive: Demands computational resources

Fig 4.1.2: Chunking techniques for RAG

Text Splitter

Let's first understand the base class used by all Langchain splitters. The **`_merge_splits`** method of the **`TextSplitter`** class is responsible for combining smaller pieces of text into medium-sized chunks. It takes a sequence of text splits and a separator and then iteratively merges these splits into chunks, ensuring that the combined size of the chunks is within specified limits.

The method uses **`chunk_size`** and **`chunk_overlap`** to determine the maximum size of the resulting chunks and their allowed overlap. It also considers factors such as the length of the separator and whether to strip whitespace from the chunks.

The logic maintains a running total of the length of the chunks and the separator. As splits are added to the current chunk, the method checks if adding a new split would exceed the specified chunk size. If so, it creates a new chunk, considering the chunk overlap, and removes splits from the beginning to meet size constraints.

This process continues until all splits are processed, resulting in a list of merged chunks. The method ensures that the chunks are within the specified size limits and handles edge cases, such as chunks longer than the specified size, by issuing a warning.

Key parameters you'll need to remember:

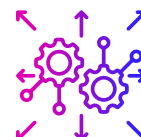


Splits

The smaller pieces of text that need to be combined.

Separator

The character(s) used to separate the splits when they're merged (e.g., space, newline).



Chunk Size



The maximum allowed size of each resulting chunk.

Chunk Overlap

The amount of overlap that you want between consecutive chunks. Remember that this will result in the duplication of data across chunks.



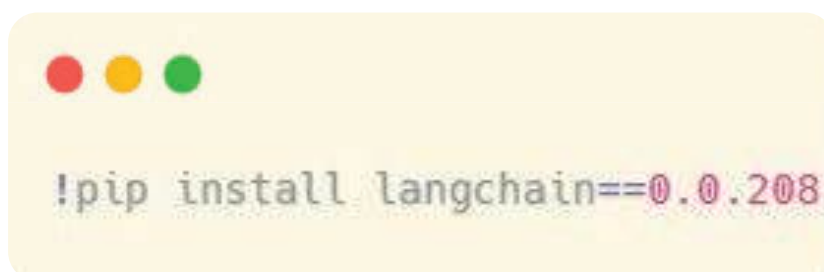
Character Splitter

Langchain's `CharacterTextSplitter` class is responsible for breaking down a given text into smaller chunks. It uses a separator such as “`\n`” to identify points where the text should be split.

- Pros: Easy and simple
- Cons: Very rigid and doesn't take into account the structure of your text

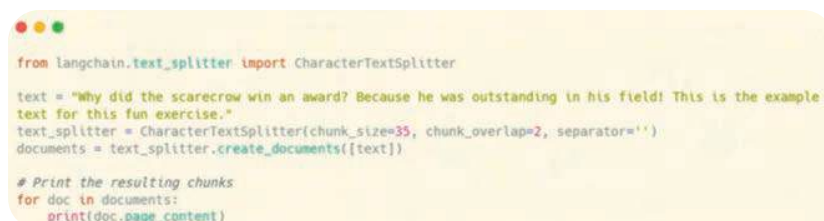
The method first splits the text using the specified separator and then merges the resulting splits into a list of chunks. The size of these chunks is determined by parameters like `chunk_size` and `chunk_overlap`, which are defined in the parent class `TextSplitter`.

Before you experiment with the code snippet (as shown below), make sure the dependency is installed. (See Fig 4.1.3).



```
!pip install langchain==0.0.208
```

Fig 4.1.3: Code snippet for installing langchain

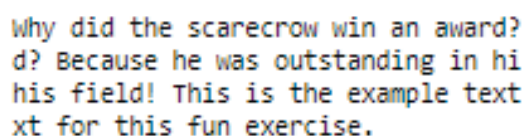


```
from langchain.text_splitter import CharacterTextSplitter

text = "Why did the scarecrow win an award? Because he was outstanding in his field! This is the example text for this fun exercise."
text_splitter = CharacterTextSplitter(chunk_size=35, chunk_overlap=2, separator='')
documents = text_splitter.create_documents([text])

# Print the resulting chunks
for doc in documents:
    print(doc.page_content)
```

Fig 4.1.4 Code snippet for `CharacterTextSplitter`



```
Why did the scarecrow win an award?
d? Because he was outstanding in hi
his field! This is the example text
xt for this fun exercise.
```

Fig 4.1.5: Output of the character text splitting technique

Recursive Character Splitter

Langchain's ***RecursiveCharacterTextSplitter*** class is designed to break down a given text into smaller chunks by recursively attempting to split it using different separators. This class is particularly useful when a single separator may not be sufficient to identify the desired chunks.

The method starts by trying to split the text using a list of potential separators specified in the ***_separators*** attribute. It iteratively checks each separator to find the one that works for the given text. If a separator is found, the text is split, and the process is repeated recursively on the resulting chunks until the chunks are of a manageable size.

The separators are listed in descending order of preference, and the method attempts to split the text using the most specific ones first. These are some common separators:

- `"\n\n"`: Double new line, or most commonly, paragraph breaks
- `"\n"`: New lines
- `" "`: Spaces

If a separator is found, it proceeds to split the text recursively.

The resulting chunks are then merged and returned as a list. The size of the chunks is determined by parameters like ***chunk_size*** and ***chunk_overlap*** defined in the parent class ***TextSplitter***. This approach allows for a more flexible and adaptive way of breaking down a text into meaningful sections.

The simple code snippet uses the ***RecursiveCharacterTextSplitter*** class (following the default separators). See Fig 4.1.6 for the code snippet and Fig 4.1.7 for the output.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text = (
    "Why did the scarecrow win an award? Because he was outstanding in his field! "
    "This is the example text for this fun exercise. Another sentence to add more variety. "
    "Splitting text can be fun and educational."
)

# Configuration 1: Small chunk size, no overlap
splitter1 = RecursiveCharacterTextSplitter(chunk_size=50, chunk_overlap=0)
documents1 = splitter1.create_documents([text])

# Configuration 2: Larger chunk size, no overlap
splitter2 = RecursiveCharacterTextSplitter(chunk_size=100, chunk_overlap=0)
documents2 = splitter2.create_documents([text])

# Configuration 3: Small chunk size, with overlap
splitter3 = RecursiveCharacterTextSplitter(chunk_size=50, chunk_overlap=10)
documents3 = splitter3.create_documents([text])

# Configuration 4: Larger chunk size, with overlap
splitter4 = RecursiveCharacterTextSplitter(chunk_size=100, chunk_overlap=20)
documents4 = splitter4.create_documents([text])

def print_documents(documents, config_number):
    print(f"\nConfiguration {config_number}:")
    for i, doc in enumerate(documents, 1):
        print(f"Chunk {i}: {doc.page_content}")

print_documents(documents1, 1)
print_documents(documents2, 2)
print_documents(documents3, 3)
print_documents(documents4, 4)
```

Fig 4.1.6: Code snippet for RecursiveCharacterTextSplitter

CONFIGURATION 1:

Chunk 1: Why did the scarecrow win an award? Because he was

Chunk 2: outstanding in his field! This is the example

Chunk 3: text for this fun exercise. Another sentence to

Chunk 4: add more variety. Splitting text can be fun and

Chunk 5: educational.

CONFIGURATION 2:

Chunk 1: Why did the scarecrow win an award? Because he was outstanding in his field! This is the example

Chunk 2: text for this fun exercise. Another sentence to add more variety. Splitting text can be fun and

Chunk 3: educational.

CONFIGURATION 3:

Chunk 1: Why did the scarecrow win an award? Because he was

Chunk 2: he was outstanding in his field! This is the

Chunk 3: is the example text for this fun exercise.

Chunk 4: exercise. Another sentence to add more variety.

Chunk 5: variety. Splitting text can be fun and

Chunk 6: fun and educational.

CONFIGURATION 4:

Chunk 1: Why did the scarecrow win an award? Because he was outstanding in his field! This is the example

Chunk 2: This is the example text for this fun exercise. Another sentence to add more variety. Splitting

Chunk 3: variety. Splitting text can be fun and educational.

Fig 4.1.7: Output of recursive character splitting technique

Sentence Splitter

Character splitting poses an issue as it tends to cut sentences midway. Despite attempts to address this using chunk size and overlap, sentences can still be cut off prematurely. Let's explore a novel approach that considers sentence boundaries instead.

The **SpacySentenceTokenizer** takes a piece of text and divides it into smaller chunks, with each chunk containing a certain number of sentences. It uses the Spacy library to analyze the input text and identify individual sentences.

The method allows you to control the size of the chunks by specifying the stride and overlap parameters. The stride determines how many sentences are skipped between consecutive chunks, and the overlap determines how many sentences from the previous chunk are included in the next one.

Follow the code snippet below to understand how you can implement it yourself. Before you run the code snippet, you must have the following dependencies installed. (See Fig 4.1.8)

```
! pip install spacy
! python -m spacy download en_core_web_sm
```

Fig 4.1.8: Code snippet to install dependencies

Refer to Fig 4.1.9 for the code snippet for using the sentence splitting technique. Fig 4.1.10 shows the output.

```
import spacy

class SpacySentenceTokenizer:
    def __init__(self, stride, overlap):
        self.stride = stride
        self.overlap = overlap
        self.nlp = spacy.load('en_core_web_sm')

    def create_documents(self, text):
        doc = self.nlp(text)
        sentences = [sent.text for sent in doc.sents]
        chunks = []

        start = 0
        while start < len(sentences):
            end = start + self.stride
            chunk = " ".join(sentences[start:end])
            chunks.append(chunk)
            start += self.stride - self.overlap

        return chunks

# example text
text = {
    "Why did the scarecrow win an award? Because he was outstanding in his field! "
    "This is the example text for this fun exercise. Another sentence to add more variety."
    "Splitting text can be fun and educational."
}

# Configuration 1: Stride of 2 sentences, overlap of 0 sentences
tokenizer1 = SpacySentenceTokenizer(stride=2, overlap=0)
documents1 = tokenizer1.create_documents(text)

# Configuration 2: A Stride of 3 sentences, overlap of 1 sentence
tokenizer2 = SpacySentenceTokenizer(stride=3, overlap=1)
documents2 = tokenizer2.create_documents(text)

def print_documents(documents, config_number):
    print(f"\nConfiguration {config_number}:")
    for i, doc in enumerate(documents, 1):
        print(f"Chunk {i}: {doc}")

# Results
print_documents(documents1, 1)
print_documents(documents2, 2)
```

Fig 4.1.9: Code snippet for using the sentence splitting technique

CONFIGURATION 1:

Chunk 1: Why did the scarecrow win an award? Because he was outstanding in his field!

Chunk 2: This is the example text for this fun exercise. Another sentence to add more variety.

Chunk 3: Splitting text can be fun and educational.

CONFIGURATION 2:

Chunk 1: Why did the scarecrow win an award? Because he was outstanding in his field! This is the example text for this fun exercise.

Chunk 2: This is the example text for this fun exercise. Another sentence to add more variety. Splitting text can be fun and educational.

Chunk 3: Splitting text can be fun and educational.

Fig 4.1.10: Output of the sentence splitting technique

Semantic Splitting

If you think about it, all the previous methods have constraints on the chunk size and don't take into account the semantics of the text, too. The semantic splitting approach takes the context of the text into consideration – relying on the idea that embeddings that represent strings will be able to infer the “contextual” relationship between chunks.

The ***SimilarSentenceSplitter*** (in the code snippet) takes a piece of text and divides it into groups of sentences based on their similarity. It utilizes a similarity model to measure how similar each sentence is to its neighboring sentences. The method uses a sentence splitter to break the input text into individual sentences.

The goal is to create groups of sentences where each group contains related sentences according to the specified similarity model. The method starts with the first sentence in the first group and then iterates through the remaining sentences.

It decides whether to add a sentence to the current group based on its similarity to the previous sentence.

The ***group_max_sentences*** parameter controls the maximum number of sentences allowed in each group. If a group reaches this limit, a new group is started. Additionally, a new group is initiated if the similarity between consecutive sentences falls below a specified ***similarity_threshold***.

In simpler terms, this method organizes a text into clusters of sentences, where sentences within each cluster are considered similar to each other. It's useful for identifying coherent and related chunks of information within a larger body of text.

Go through the code snippet below to understand how semantic splitting works. Before you run the code snippet, you'll need to install one more dependency. (See Fig 4.1.11).



```
!pip install sentence-transformers
```

Fig 4.1.11: Code snippet to install dependency

Refer to Fig 4.1.12 for the code snippet. Fig 4.1.13 shows the output of semantic splitting.

```
from sentence_transformers import SentenceTransformer, util

class SimilarSentenceSplitter:
    def __init__(self, group_max_sentences, similarity_threshold):
        self.group_max_sentences = group_max_sentences
        self.similarity_threshold = similarity_threshold
        self.model = SentenceTransformer('all-MiniLM-L6-v2')

    def create_documents(self, text):
        sentences = [sent.strip() + '.' for sent in text.split('. ') if sent]
        embeddings = self.model.encode(sentences, convert_to_tensor=True)
        chunks, current_chunk = [], [sentences[0]]

        for i in range(1, len(sentences)):
            if len(current_chunk) >= self.group_max_sentences or util.pytorch_cos_sim(embeddings[i-1],
embeddings[i]).item() < self.similarity_threshold:
                chunks.append(" ".join(current_chunk))
                current_chunk = [sentences[i]]
            else:
                current_chunk.append(sentences[i])

        if current_chunk:
            chunks.append(" ".join(current_chunk))
        return chunks

# Example usage
text = (
    "Why did the scarecrow win an award? Because he was outstanding in his field! "
    "This is the example text for this fun exercise. Another sentence to add more variety. "
    "Splitting text can be fun and educational. Here's another sentence. And one more to check the "
    "clustering."
)

# Configuration 1: Max 3 sentences per group, similarity threshold 0.8
splitter1 = SimilarSentenceSplitter(group_max_sentences=3, similarity_threshold=0.8)
documents1 = splitter1.create_documents(text)

# Configuration 2: Max 2 sentences per group, similarity threshold 0.5
splitter2 = SimilarSentenceSplitter(group_max_sentences=2, similarity_threshold=0.2)
documents2 = splitter2.create_documents(text)

# Print the results of each configuration
def print_documents(documents, config_number):
    print(f"\nConfiguration {config_number}:")
    for i, doc in enumerate(documents, 1):
        print(f"Chunk {i}: {doc}")

print_documents(documents1, 1)
print_documents(documents2, 2)
```

Fig 4.1.12: Code snippet for semantic splitting



CONFIGURATION 1:

Chunk 1: Why did the scarecrow win an award? Because he was outstanding in his field! This is the example text for this fun exercise.

Chunk 2: Another sentence to add more variety.

Chunk 3: Splitting text can be fun and educational.

Chunk 4: Here's another sentence.

Chunk 5: And one more to check the clustering..

CONFIGURATION 2:

Chunk 1: Why did the scarecrow win an award? Because he was outstanding in his field! This is the example text for this fun exercise.

Chunk 2: Another sentence to add more variety. Splitting text can be fun and educational.

Chunk 3: Here's another sentence.

Chunk 4: And one more to check the clustering..

Fig 4.1.13: Output of semantic splitting technique

Document Specific Splitting

Until now, we've dealt with sentences that are structured in nature. But what about documents that contain tables, code snippets, and more? [Unstructured](#), with its diverse document type support and flexible partitioning strategies, offers several benefits for reading documents efficiently. Let's look at how this works.

Supports All Major Document Types

Unstructured supports a wide range of document types, including .pdf, .docx, .doc, .odt, .pptx, .ppt, .xlsx, .csv, .tsv, .eml, .msg, .rtf, .epub, .html, .xml, .png, .jpg, and .txt files. This ensures users can seamlessly work with different file formats within a unified framework.

Adaptive Partitioning

The "auto" strategy in Unstructured provides an adaptive approach to partitioning. It automatically selects the most suitable partitioning strategy based on the document's characteristics. This feature simplifies the user experience and optimizes document processing without the need for manual intervention in selecting partitioning strategies.

Specialized Strategies for Varied Use Cases

Unstructured provides specific strategies for different needs. The "fast" strategy quickly extracts information using traditional NLP techniques, "**hi_res**" ensures precise classification using detectron2 and document layout, and "**ocr_only**" is designed specifically for Optical Character Recognition in image-based files. These strategies accommodate various use cases, offering users flexibility and precision in document-processing workflows.

Unstructured's comprehensive document type support, adaptive partitioning strategies, and customization options make it a powerful tool for efficiently reading and processing a diverse range of documents.

Let's go through a code snippet to look at how it partitions the [Gemini 1.5 technical report](#). (See Fig 4.1.14).

```

elements = partition_pdf(
    filename=filename,

    # Unstructured Helpers
    strategy="hi_res",
    infer_table_structure=True,
    model_name="yolox"
)

Output:
[<unstructured.documents.elements.Image at 0x2acfc24d0>,
 <unstructured.documents.elements.Title at 0x2d4562c50>,
 <unstructured.documents.elements.NarrativeText at 0x2d4563b50>,
 <unstructured.documents.elements.NarrativeText at 0x2d4563350>,
 <unstructured.documents.elements.Title at 0x2d4560b90>,
 <unstructured.documents.elements.NarrativeText at 0x2d4562350>,
 <unstructured.documents.elements.NarrativeText at 0x2d4561b10>,
 <unstructured.documents.elements.NarrativeText at 0x2d4562410>,
 <unstructured.documents.elements.NarrativeText at 0x2d45620d0>,
 <unstructured.documents.elements.Header at 0x2d4562110>,
 <unstructured.documents.elements.NarrativeText at 0x2d4560a50>,
 <unstructured.documents.elements.Title at 0x2a58e3090>,
 <unstructured.documents.elements.Image at 0x2d4563d90>,
 <unstructured.documents.elements.FigureCaption at 0x2d4563c90>,
 <unstructured.documents.elements.NarrativeText at 0x2d4563150>,
 <unstructured.documents.elements.NarrativeText at 0x2d4562290>,
 <unstructured.documents.elements.Footer at 0x2d4563e90>,
 <unstructured.documents.elements.Header at 0x2d4562790>,
 <unstructured.documents.elements.Table at 0x2d4561ed0>,
 <unstructured.documents.elements.Title at 0x2a7efeal0>,
 .....
 ]

```

Fig 4.1.14: Code snippet for partitioning using Unstructured

